# Computer Networks

**ASSIGNMENT 1**

| | | | |
|---|---|---|---|
| **Name** | Machani Sravani | **Register No.** | 1924260126 |
| **Submission Date** | 02-11-2026 | **Course** | Computer Networks (CSA0708) |
| **GitHub Repository** | https://github.com/MachaniSravani/CN-lab | **Nature / Duration** | Individual / 1 Week |

**CO1:** Apply the functions of the OSI layers in enabling reliable data transmission across networks – uses of networks, network hardware and software, components, types of connections, topologies, protocols and standards, the ISO/OSI reference model, transmission media, wireless communication, Ethernet configuration, an introduction to 5G networks, and error detection, error correction and data-link control.

**Assignment Title:** Make the Data of a Smart Manufacturing Plant Arrive Correctly and On Time

(Total Marks – 50)

**Nature of Assignment:** Individual

**Duration:** 1 Week

**Submission Date:** 02-11-2026

## TASK 1 – Size the Traffic Before Choosing Anything

### 1. Traffic of one line and of the plant

| Source | Working | Traffic per line |
|---|---|---|
| Robot controllers | 40 × (200 B × 8 bits ÷ 0.002 s) = 40 × 0.8 Mbps | 32.00 Mbps |
| Vibration sensors | 600 × 50 B/s × 8 = 240,000 bit/s | 0.24 Mbps |
| CCTV cameras | 24 × 4 Mbps (continuous) | 96.00 Mbps |
| Office users | 150 × 10 % = 15 active × 2 Mbps | 30.00 Mbps |
| **Total – 1 line** | 32 + 0.24 + 96 + 30 | **158.24 Mbps** |
| **Total – plant (8 lines)** | 8 × 158.24 | **1,265.92 Mbps ≈ 1.27 Gbps** |

### 2. Biggest sender vs strictest delay

- **Most data:** CCTV, 96 Mbps = 60.7 % of the line. **Strictest delay:** robot controllers, 10 ms (only 32 Mbps = 20.2 %). They are **not the same**: the heaviest source tolerates delay, the most delay-critical source is light.

### 3. Is a single 1 Gbps uplink enough?

- One line: 158.24 Mbps = **15.8 %** of 1 Gbps – enough. Whole plant: 1,265.92 Mbps = **126.6 %** – **not enough** (and a safe design keeps links below ~70 % = 700 Mbps). Needs a 10 Gbps uplink (12.7 % load) or a link aggregation of several 1 Gbps links.
- **Shared queue:** when offered load is above link speed the queue stays full. A 1 MB buffer needs 1 MB × 8 ÷ 1 Gbps = **8 ms** to drain, so a robot command waits behind CCTV frames (head-of-line blocking) and, with propagation and processing, misses its 10 ms deadline; once the buffer overflows commands are also dropped. Fix: separate VLAN and a strict-priority (DSCP EF) queue for robot traffic.

### 4. Delay-versus-volume comparison and ranking by harm of a 100 ms delay

| Rank | Traffic | Volume (share) | Delay budget | Harm of a 100 ms delay – reason |
|---|---|---|---|---|
| 1 | Robot commands | 32 Mbps (20.2 %) | 10 ms | 100 ms = 50 missed 2 ms commands, 10× the deadline → collisions, scrap, safety stop. |
| 2 | CCTV video | 96 Mbps (60.7 %) | ~100–500 ms | ≈3 video frames late at 30 fps; jitter/freeze for live monitoring, but buffers hide most of it. |
| 3 | Vibration sensors | 0.24 Mbps (0.15 %) | ~1 s | 100 ms is only 10 % of the 1 s reporting period; alarm is slightly late. |
| 4 | Office / ERP / e-mail | 30 Mbps (19.0 %) | ≥ 150 ms | Humans barely notice 100 ms; e-mail is not time-sensitive. |

### Alternative Concept Analysis – motion-triggered CCTV (5 % of the time)

- CCTV = 24 × 4 Mbps × 0.05 = **4.8 Mbps**. Line = 32 + 0.24 + 4.8 + 30 = **67.04 Mbps**; plant = 8 × 67.04 = **536.32 Mbps** (53.6 % of 1 Gbps, a 57.6 % reduction).

- **Decision:** the 1 Gbps uplink is **no longer the bottleneck** (53.6 % < 70 %), but little room remains for 1,500 devices or 5 plants, so a 10 Gbps backbone is still recommended. Costs of the alternative: local storage and motion detection, and no continuous live view.

## TASK 2 – Count the Domains in the Contractor's Drawing

### 1. Domain count

- **Collision domains = 17:** each hub with its 8 PCs and its uplink to the switch is one shared segment (3); SW1–router G0/0 (1); router G0/1–SW2 (1); each of the 12 IP-phone ports on SW2 is its own full-duplex link (12). 3 + 1 + 1 + 12 = **17**.
- **Broadcast domains = 2:** switches and hubs forward broadcasts; only the router splits them – G0/0 side (24 PCs) and G0/1 side (12 phones).

### 2. Device layers and why a hub cannot stop collisions

| Device | OSI layer | Why |
|---|---|---|
| Hub | 1 Physical | Repeats electrical bits to every port; does not read frames or MAC addresses, so all ports share one collision domain (CSMA/CD). |
| Switch | 2 Data link | Reads MAC addresses, learns a MAC table, buffers frames; every port is a separate collision domain. |
| Router | 3 Network | Reads IP addresses, forwards between networks; blocks broadcasts. |
| PC / IP phone | 1–7 (all) | End devices implement the full stack. |

### 3. Point-to-point or multipoint

- **Point-to-point:** SW1–G0/0, G0/1–SW2, SW2–each phone, each hub–SW1 cable. **Multipoint (shared):** each hub segment, where 8 PCs share one medium.

### 4. Standards table

| Technology | Standard | Body |
|---|---|---|
| Ethernet | IEEE 802.3 | IEEE (Institute of Electrical and Electronics Engineers) |
| Wi-Fi | IEEE 802.11 | IEEE (interoperability certified by the Wi-Fi Alliance) |
| IPv4 | RFC 791 | IETF (Internet Engineering Task Force) |
| OSI model | ISO 7498 | ISO (International Organization for Standardization) |
| 5G | 3GPP Release 15+ / IMT-2020 | 3GPP specifications; ITU defines IMT-2020 requirements |

Buy only standard-compliant equipment: it interoperates across vendors, avoids lock-in, is tested for safety/EMC and can be replaced or expanded without redesign.

### Alternative Concept Analysis – replace the three hubs with unmanaged switches

- Collision domains: 24 PC links + 3 switch-to-SW1 links + 1 + 1 + 12 = **41**. Broadcast domains stay **2**.
- **Fixes:** collisions on the PC segments and shared bandwidth (each PC gets its own full-duplex 100 Mbps instead of sharing it with 7 others). **Does not fix:** broadcast traffic (still one 24-PC

domain), no VLANs/QoS/security/monitoring or loop protection (unmanaged), and no layer-3 separation.

## TASK 3 – Read a Captured Frame Layer by Layer

### 1. Layer-by-layer reading

| Capture line | OSI layer | PDU | Meaning |
|---|---|---|---|
| Ethernet II | 2 Data link | Frame | Dst/Src MAC; Type 0x0800 = IPv4 inside |
| IPv4 | 3 Network | Packet | 10.10.1.25 → 10.10.9.4, TTL 61, Protocol 6 = TCP |
| TCP | 4 Transport | Segment | Port 51522 → 502, Seq 1201, Ack 88, Len 12 |
| Data (Modbus write) | 7 Application | Data / message | 12-byte command |

- **Routers crossed:** each router lowers TTL by 1 → 64 − 61 = **3 routers**.

### 2. Bytes on the wire and overhead (Ethernet 14 + IP 20 + TCP 20 + FCS 4 = 58 B)

| Packet | Total on wire | Overhead | Efficiency |
|---|---|---|---|
| 12-byte Modbus command | 14+20+20+12+4 = **70 B** | 58 B = **82.9 %** | 17.1 % |
| 1,400-byte camera packet | 1,400 + 58 = **1,458 B** | 58 B = **3.98 %** | 96.0 % |

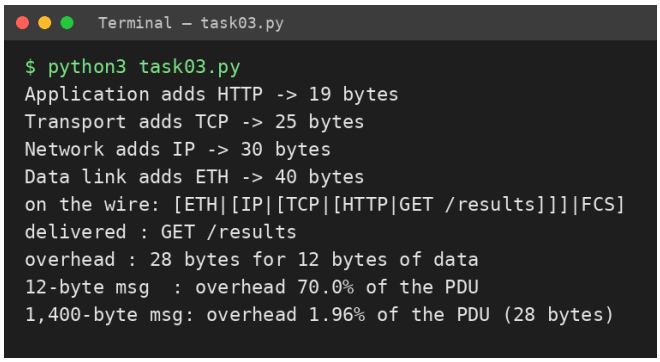
- Small commands waste ~83 % of the wire; large packets only ~4 %. Even the 200-byte command of Task 1 carries 58/258 = 22.5 % overhead, so 40 robots really load 40 × 500 × 258 × 8 = **41.28 Mbps**, not 32 Mbps.

### 3. Fault → responsible layer

- (a) Wrong MAC address → **Layer 2 Data link** (NIC discards the frame). (b) TTL reaches 0 → **Layer 3 Network** (router drops it, sends ICMP Time Exceeded). (c) Closed destination port → **Layer 4 Transport** (host answers TCP RST).
- **TCP/IP mapping:** Application layer = OSI 5–7 (session, presentation, application); Transport = 4; Internet = 3; Network access = 1–2 (physical + data link).

### Complete the Code – TODO 1

```
if tag == "ETH":  pdu = "[" + tag + "|" + pdu + "|FCS]"
else:             pdu = "[" + tag + "|" + pdu + "]"
print(f"{layer} adds {tag} -> {len(pdu)} bytes")
--- output ---
Application adds HTTP -> 19 bytes   |   Transport adds TCP -> 25 bytes
Network adds IP -> 30 bytes         |   Data link adds ETH -> 40 bytes
on the wire: [ETH|[IP|[TCP|[HTTP|GET /results]]]|FCS]
delivered : GET /results            |   overhead : 28 bytes for 12 bytes of data
```




Output matches the expected output. With a 1,400-byte message: 28 B overhead = **1.96 %** of the PDU, against **70 %** for the 12-byte message.

### Alternative Concept Analysis – command directly in Ethernet (EtherType 0x8892)

- Removing IP + TCP removes 40 header bytes, but Ethernet needs ≥ 46 B payload, so 12 B is padded: frame = 14 + 46 + 4 = **64 B**. Real saving = 70 − 64 = **6 B/command** (40 B only if the data ≥ 46 B).
- 100 Mbps link (adding 20 B preamble + gap): TCP/IP 90 B → **138,889 cmd/s**; raw Ethernet 84 B → **148,810 cmd/s**; frees ≈ **9,921 cmd/s (+7.1 %)**.
- **Lost by giving up IP:** routing (stays inside one broadcast domain, cannot reach the head office), IP addressing and firewalls/VPN, TCP reliability (ACK, retransmission, ordering, flow control) and ordinary tools/interoperability.

## TASK 4 – Choose a Medium for Each Run

### 1. Medium for each run

| Run | Medium | Deciding reason |
|---|---|---|
| (a) Robot cell by welders, 85 m | Cat6A S/FTP | 85 m < 100 m copper limit; foil + braid shielding rejects welding EMI (fibre if SNR still too low). |
| (b) Office → shop floor, 140 m | Multimode fibre (OM3/OM4) | 140 m > 100 m copper channel limit; fibre also isolates the office from electrical noise. |
| (c) Shop floor → data centre, 550 m | Single-mode fibre | Beyond 10G multimode reach (~300–400 m); single-mode reaches 10 km+ and takes 25/100G later. |
| (d) Plant → head office, 400 km | Leased line (carrier fibre) | Beyond the reach of unrepeated fibre (~10–80 km); carrier amplifiers/regenerators needed. |

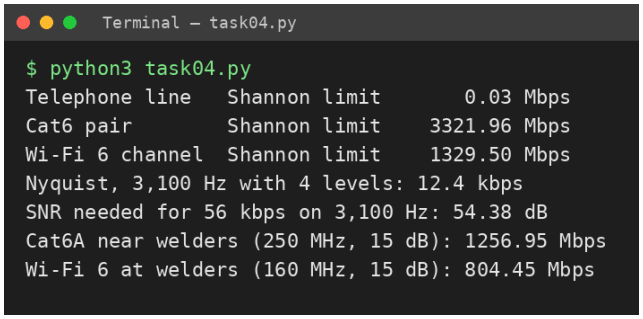
- **The trap:** copper Ethernet (100 m) cannot be used for (b), (c) or (d). Run (b) at 140 m looks only "a little over" but is still out; (c) fits 1G multimode but not 10G; **(d) cannot use copper at all – nor self-installed fibre –** so a carrier line is required.

### 2. Shannon capacity, $C = B \cdot \log_2(1 + SNR)$

- 250 MHz, 40 dB (ratio 10,000): $C = 250 \times 10^6 \times \log_2(10{,}001) =$ **3,321.96 Mbps**.
- 250 MHz, 15 dB (ratio 31.62): $C = 250 \times 10^6 \times \log_2(32.62) =$ **1,256.95 Mbps** – a **62 % drop**.
- **Meaning for the robot cell:** noise near the welders removes 62 % of the ceiling; 1,257 Mbps is still far above the robot need (32 Mbps) but only 1.26× a 1 Gbps link, and real links reach 60–70 % of Shannon, so errors, CRC failures and retransmissions rise. Hence shielded Cat6A (or fibre) routed away from welding cables.
- **Measured speed is always below Shannon** because the limit assumes ideal coding; real links use finite modulation, lose power to attenuation/crosstalk, need an SNR margin, and spend bits on preamble, headers, FCS, gaps and retransmissions.

### Complete the Code – TODO 1 and 2 (executed, Python 3)

```
def nyquist(bandwidth_hz, levels):  return 2 * bandwidth_hz * math.log2(levels)
def shannon(bandwidth_hz, snr_db):  snr = 10 ** (snr_db / 10);  return bandwidth_hz *
math.log2(1 + snr)
--- output ---
Telephone line   Shannon limit       0.03 Mbps
Cat6 pair        Shannon limit    3321.96 Mbps
Wi-Fi 6 channel  Shannon limit    1329.50 Mbps
Nyquist, 3,100 Hz with 4 levels: 12.4 kbps
```



- **56 kbps on 3,100 Hz:** 56,000/3,100 = 18.06 bit/Hz → SNR = 2^18.06 − 1 = 274,129 → **54.4 dB**. Not realistic: a telephone line gives ~30–35 dB (≈ 30–35 kbps; 0.03 Mbps above); 56k modems worked only with a digital trunk on the server side.

### Alternative Concept Analysis – Wi-Fi 6 in the robot cell

- 160 MHz, 15 dB: $C = 160 \times 10^6 \times \log_2(32.62)$ = **804.45 Mbps** (36 % below the cable). Capacity is plenty, but the problem is **latency**: CSMA/CA back-off, retries and interference give no 10 ms guarantee (tails of 100 ms+). **Not acceptable** for 10 ms robot commands; use shielded cable or 5G URLLC, and keep Wi-Fi 6 for monitoring/office use.

## TASK 5 – Wi-Fi 6 or 5G for Moving Equipment?

### 1. Hand-overs on a 1.2 km trip at 2 m/s (trip time = 1,200 ÷ 2 = 600 s)

| Measure | Wi-Fi (30 m APs, 150 ms) | 5G (300 m cell, 20 ms) |
|---|---|---|
| Cells along the route | 1,200 ÷ 30 = 40 | 1,200 ÷ 300 = 4 |
| Hand-overs per trip | **39** (≈ one every 15 s) | **3** (one every 150 s) |
| Time not reachable per trip | 39 × 150 ms = **5.85 s** (0.975 %) | 3 × 20 ms = **60 ms** (0.01 %) |
| 10 ms commands missed per trip | 585 | 6 |
| Distance travelled "blind" (2 m/s) | 11.7 m | 0.12 m |

### 2. 5G service type needed

| Traffic | 5G service | Reason |
|---|---|---|
| AGVs (and robots) | **URLLC** | 10 ms deadline, very high reliability, mobility. |
| CCTV cameras | **eMBB** | 96 Mbps per line of continuous video – high throughput. |
| Sensors (600 × 50 B/s) | **mMTC** | Very many devices, tiny messages, battery life. |

### 3. Where Bluetooth, Zigbee or LoRa beat Wi-Fi/5G

- **Bluetooth (BLE):** worker badges, handheld scanners, tool tags, headsets – ≤ 10–50 m, low power. **Zigbee:** mesh of battery-powered sensors, switches and lighting – ≈ 250 kbps, 10–100 m. **LoRa:** far, slow, battery-powered nodes such as yard, tank or warehouse monitoring – kilometres, years of battery, few kbps.

### 4. Could Wi-Fi 6 with fast roaming (802.11r) meet the deadline?

- 802.11r pre-shares keys so a hand-over drops from ~150 ms to roughly 20–50 ms. Even at 30 ms: 39 × 30 ms = 1.17 s, ≈ 117 missed commands per trip – each hand-over still exceeds 10 ms. **Remaining risk:** contention with 150 office users, welding interference, retries and tail latency, ping-pong roaming, metal-reflection dead spots, and no hard QoS guarantee. Acceptable for non-critical data, **not for "never miss a command"**.

### Alternative Concept Analysis – LoRaWAN for the sensors

- Load = 600 × 50 B × 8 = **240 kbps** versus ~5 kbps per gateway → **48× too much** (4,800 %). One gateway supports only 5,000 ÷ 400 bit/s = **12.5 sensors** at 50 B/s; 600 sensors would need 48 gateways (and LoRa duty-cycle limits make it worse).
- **Decision:** keep the continuous 50 B/s vibration sensors on Wi-Fi/5G mMTC (or wired). Move slow sensors (e.g. temperature, 50 B every 60 s = 6.7 bit/s each → ~750 per gateway) to LoRaWAN.

## TASK 6 – Interpret an Ethernet Port Report

### 1. Reading the report

- **Late collision (412):** a collision detected after the first 512 bits (64 bytes) of a frame – too late for the NIC to retransmit; caused by a duplex mismatch or an over-long segment/too many hubs. **CRC error (1,907):** the received FCS does not match – the frame is damaged or truncated by noise or a collision.
- **Combination:** classic **duplex mismatch**. The PC is forced to 100 Mb/s **half** duplex with auto-negotiation off; the switch end is operating full duplex and transmits whenever it likes. The half-duplex PC sees late collisions and CRC/runt frames (2,381 rx errors), and the full-duplex switch side sees corrupted frames too – the link is "up" (Link detected: yes) but slow.

### 2. Likely cause and exact fix

- Auto-negotiation is off on the PC while the gigabit switch port is auto (a forced end cannot advertise duplex). Fix by returning both ends to auto-negotiation (1000BASE-T requires it) and verifying the cable:

```
# Linux
sudo ethtool -s eth0 autoneg on                # or: speed 1000 duplex full autoneg on
ethtool eth0 | grep -E "Speed|Duplex|Auto"      # expect 1000Mb/s, Full, on
ethtool -S eth0 | grep -E "err|collision"       # counters must stop rising
# Windows: Device Manager > adapter > Properties > Advanced > Speed & Duplex > Auto Negotiation
```

### 3. MAC addresses

- **01:00:5e:0a:01:02:** first byte 0x01 → least-significant bit = 1 → a **group (multicast)** address, not one device (01:00:5e is the IPv4-multicast prefix; 0a:01:02 maps to 224.10.1.2).
- **3c:52:82:4e:11:07:** the first three bytes **3c:52:82** are the **OUI** assigned by IEEE, identifying the NIC manufacturer (look it up in the IEEE registry/Wireshark); 4e:11:07 is the unique serial of that card. First byte 0x3c has bits 0 and 1 = 0 → unicast, globally administered.

### 4. Jumbo frames (9,000 bytes) for the CCTV storage server

- Efficiency: 1,460 B data per 1,518 B frame = 96.2 % → 8,960 B per 9,018 B = 99.4 % (only +3.2 % bandwidth), but **6.1× fewer frames** (8,960 ÷ 1,460), so far fewer interrupts and CPU load – **it helps** the storage server.
- **Must also change:** MTU 9000 on every NIC, switch port and router interface along the path (e.g. `ip link set eth0 mtu 9000`); otherwise frames are dropped or fragmented. Keep jumbo in the CCTV/storage VLAN only – a 9,000 B frame takes 72 µs at 1 Gbps (720 µs at 100 Mbps) and would delay robot packets.

### Alternative Concept Analysis – ring of industrial switches vs star

| | **Ring with fast recovery (e.g. MRP / RSTP)** | **Star of switches** |
|---|---|---|
| Recovery time after one cut | ≈ 10–200 ms (MRP profiles); RSTP ≈ 1–2 s – traffic re-routes the other way round | None automatic: stays down until the cable is repaired (hours); dual uplinks + RSTP ≈ 1–2 s |

| | **Ring with fast recovery (e.g. MRP / RSTP)** | **Star of switches** |
|---|---|---|
| Devices that lose connection | **None** permanently – ring becomes a line; only a short outage during recovery | Drop cable cut: that **1 device**. Uplink cut: **all devices on that switch** |
| Cost / limits | Fewer cables, redundancy built-in; recovery time grows with ring size | Simple, easy fault-finding; uplink/core are single points of failure |

## TASK 7 – Will the Receiver Notice the Error?

### 1. CRC of 1101011011 with generator 1011 ($x^3 + x + 1$): append 3 zeros → 1101011011000

```
1101 xor 1011 = 0110 -> bring down 0 -> 1100
1100 xor 1011 = 0111 -> bring down 1 -> 1111
1111 xor 1011 = 0100 -> bring down 1 -> 1001
1001 xor 1011 = 0010 -> bring down 0 -> 0100 (lead 0, xor 0000) -> 100, bring down 1 -> 1001
1001 xor 1011 = 0010 -> bring down 1 -> 0101 (lead 0) -> 101, bring down 0 -> 1010
1010 xor 1011 = 0001 -> bring down 0 -> 0010 (lead 0) -> 010, bring down 0 -> 0100 ->
remainder 100
```

**CRC = 100; transmitted frame = 1101011011100.**

- **Received 1101011011101:** the same division leaves remainder **001 ≠ 000** → the frame is **rejected** (error detected; program-confirmed).
- **Longest burst always detected:** the generator has degree 3, so every burst of **≤ 3 bits** is detected (4-bit bursts: 75 %; longer: 87.5 %). **Undetectable pattern:** any multiple of the generator, e.g. error **0000000001011** (= 1011 itself) – the remainder is 0, so it passes.

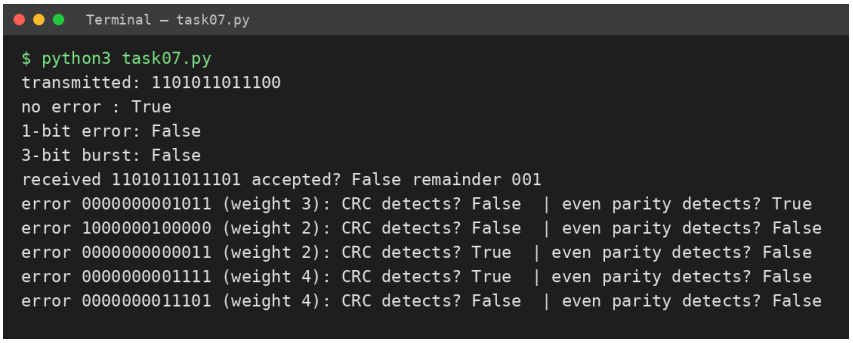
### 2. Two-dimensional parity (even) of a 4 × 4 block and the Internet checksum

| Row | c1 | c2 | c3 | c4 | Row parity |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 1 | 1 | 1 |
| 2 | 0 | 1 | 1 | 0 | 0 |
| 3 | 1 | 1 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 1 | 1 |
| Column parity | 0 | 0 | 0 | 0 | 0 |

- **Missed 4-bit pattern:** flip the four corners of a rectangle – (r1,c1), (r1,c2), (r2,c1), (r2,c2): every row and column still has even parity, so the error is accepted.
- **Internet checksum:** 4500 + 0073 + 0000 + 4000 = 8573 (no carry) → one's complement = FFFF − 8573 = **7A8C** (receiver: 8573 + 7A8C = FFFF ✓).

### Complete the Code – TODO 1 (executed, Python 3)

```
if rem[i] == "1":
    for j in range(len(divisor)): rem[i + j] = str(int(rem[i + j]) ^ int(divisor[j]))
--- output ---  transmitted: 1101011011100 | no error: True | 1-bit error: False | 3-bit burst: False
```




Program search: error pattern **0000000001011** gives "CRC detects? False"; so does 1000000100000 (two bits 7 apart, because $x^7 + 1$ is divisible by $x^3 + x + 1$).

### Alternative Concept Analysis – even parity bit vs CRC-3 (13-bit frame)

| Error pattern | Bits | Even parity | CRC-3 (1011) |
|---|---|---|---|
| 0000000000011 (adjacent) | 2 | Missed (even) | Detected |
| 1000000100000 (7 apart) | 2 | Missed (even) | Missed (multiple of G) |
| 0000000001111 | 4 | Missed (even) | Detected |
| 0000000011101 (= 1011 × 11) | 4 | Missed (even) | Missed (multiple of G) |
| 0000000001011 (= G) | 3 | Detected (odd) | Missed |

Parity catches every odd-weight error, including the 3-bit pattern that CRC misses, but misses every even-weight error. CRC-3 catches all 1-bit errors, all bursts ≤ 3 bits and most others, so it is stronger overall (3 check bits instead of 1).

## TASK 8 – Correct the Error or Ask Again?

### 1. Hamming check bits for a 16-bit word ($2^r \geq m + r + 1$)

- r = 4: $16 \geq 16 + 4 + 1 = 21$? **No**. r = 5: $32 \geq 22$ **Yes** → **5 check bits**, 21-bit codeword. Overhead = 5/16 = **31.25 %** of the data (23.8 % of the codeword).

### 2. Minimum Hamming distance of {00000, 01011, 10101, 11110}

| Pair | Distance | Pair / distance |
|---|---|---|
| 00000–01011 | 3 | 01011–10101: 4 |
| 00000–10101 | 3 | 01011–11110: 3 |
| 00000–11110 | 4 | 10101–11110: 3 |

- **d_min = 3** → detects up to d − 1 = **2** errors and corrects ⌊(d − 1)/2⌋ = **1** error.

### 3. Hamming(7,4): received 0110111 (positions 1–7 = 0,1,1,0,1,1,1)

- s1 (positions 1,3,5,7) = 0⊕1⊕1⊕1 = **1**; s2 (2,3,6,7) = 1⊕1⊕1⊕1 = **0**; s3 (4,5,6,7) = 0⊕1⊕1⊕1 = **1** → position = 1 + 0 + 4 = **5**. Flip bit 5 → **0110011** (the sent codeword); data bits (3,5,6,7) = **1011**.

### 4. FEC or ARQ?

- On a noisy radio link a retransmission costs 40 ms, longer than any real-time deadline, so **forward error correction** keeps delay bounded. Video streaming and 5G use FEC because a retransmitted packet arrives too late to be useful (video), a broadcast cannot wait for every receiver, and the radio channel is noisy (5G uses LDPC/polar codes plus HARQ).

### Complete the Code – TODO 1 (executed, Python 3)

```
s1 = c[0]^c[2]^c[4]^c[6];  s2 = c[1]^c[2]^c[5]^c[6];  s3 = c[3]^c[4]^c[5]^c[6]
pos = s1 + 2*s2 + 4*s3;  if pos: c[pos-1] ^= 1;  return pos, [c[2], c[4], c[5], c[6]]
--- output ---  codeword : 0110011 | error at None: syndrome 0, data 1011 | error at 5: syndrome 5, data 1011
```

```
Terminal — task08.py
$ python3 task08.py
codeword : 0110011
error at None: syndrome 0, data 1011
error at 5   : syndrome 5, data 1011
double error (pos 3 and 6): syndrome 5 -> data 0101 (wrong, original 1011)
0110111 -> syndrome 5 data 1011
```

- **Two bits flipped** (positions 3 and 6): syndrome = 3 ⊕ 6 = 5, so the decoder "corrects" position 5 – the wrong bit – and returns data **0101** instead of 1011. Hamming(7,4) (d = 3) corrects only one error.

### Alternative Concept Analysis – BER = $10^{-3}$, 16-bit readings

| Measure | Hamming FEC (21 bits) | Retransmission ARQ (16 bits) |
|---|---|---|
| Probability a transmission is usable | $(0.999)^{21} + 21(0.001)(0.999)^{20} = 0.99979$ | $(0.999)^{16} = 0.98412$ |
| Average transmissions | 1.0002 | 1 / 0.98412 = 1.0161 |
| Bits sent per reading | 21 × 1.0002 = **21.00** | 16 × 1.0161 = **16.26** |

| Measure | Hamming FEC (21 bits) | Retransmission ARQ (16 bits) |
|---|---|---|
| Average extra delay | 0.0002 × 40 ms = **0.008 ms** | 0.0161 × 40 ms = **0.65 ms** |
| Worst case | 40 ms (rare double error) | 40 ms+ for ≈ 1.6 % of readings |

ARQ sends 22.6 % fewer bits, but 1.6 % of readings suffer 40 ms spikes. FEC costs 31 % more bits yet keeps delay almost constant (0.008 ms mean), so FEC suits time-critical sensor data and ARQ suits bandwidth-limited, delay-tolerant data.

## TASK 9 – How Big Must the Window Be?

### 1. Timing of the 400 km, 1 Gbps link (frame = 1,500 B = 12,000 bits)

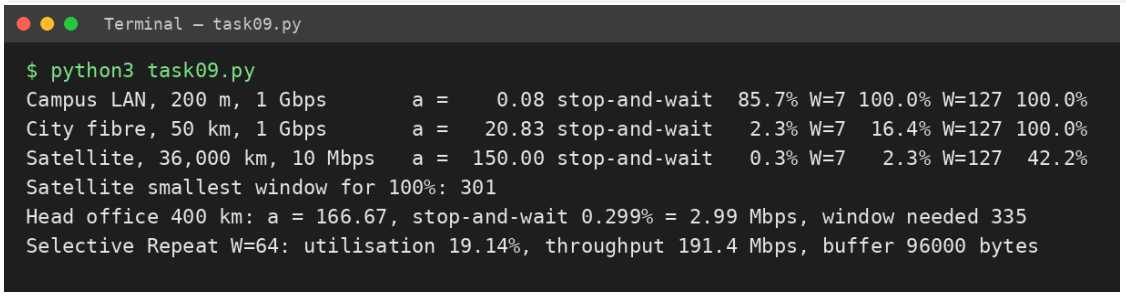
| Quantity | Working |
|---|---|
| Transmission time | $12{,}000 \div 10^9$ = **12 µs** |
| Propagation time | $400{,}000$ m $\div 2\times10^8$ m/s = **2 ms** (2,000 µs) |
| a = propagation ÷ transmission | 2,000 ÷ 12 = **166.67** |
| Stop-and-wait utilisation | 1 ÷ (1 + 2a) = 1 ÷ 334.33 = **0.299 % → 2.99 Mbps** of 1,000 Mbps |
| Smallest window for full use | $W \geq 1 + 2a = 334.33$ → **335 frames** (≈ 502.5 KB = bandwidth–delay product) |
| Sequence-number bits | Go-Back-N: $W \leq 2^k - 1 \rightarrow 2^9 - 1 = 511 \geq 335$ → **9 bits**<br>Selective Repeat: $W \leq 2^{k-1} \rightarrow 2^9 = 512 \geq 335$ → **10 bits** |

### 2. Go-Back-N vs Selective Repeat, and framing

- After one loss **Selective Repeat resends 1 frame**; Go-Back-N resends the lost frame and everything after it (up to ~335 frames). **Cost of SR:** the receiver needs a buffer for out-of-order frames (335 × 1,500 B ≈ 502.5 KB), reordering logic and per-frame ACKs/timers; the GBN receiver needs no buffer.
- **HDLC** marks a frame with the flag **01111110** at start and end, and uses **bit stuffing** (insert 0 after five consecutive 1s). **PPP** uses flag **0x7E** and **byte stuffing** (0x7E/0x7D in data become 0x7D followed by the byte XOR 0x20).

### Complete the Code – TODO 1 (executed, Python 3)

```
a = prop / tx;   return min(1, window / (1 + 2 * a)), a
--- output ---
Campus LAN, 200 m, 1 Gbps      a =    0.08 stop-and-wait  85.7% W=7 100.0% W=127 100.0%
City fibre, 50 km, 1 Gbps      a =   20.83 stop-and-wait   2.3% W=7  16.4% W=127 100.0%
Satellite, 36,000 km, 10 Mbps  a =  150.00 stop-and-wait   0.3% W=7   2.3% W=127  42.2%
```




**Smallest window for 100 % on the satellite link:** 1 + 2a = 1 + 300 = **301 frames** (W = 127 gives 127/301 = 42.2 %, matching the output).

### Alternative Concept Analysis – Selective Repeat with window 64 on the 400 km link

- Utilisation = 64 ÷ 334.33 = **19.14 %** → throughput = **191.4 Mbps** (80.9 % of the link wasted, versus 100 % with W = 335). Receiver buffer = 64 × 1,500 B = **96,000 B ≈ 96 KB** (vs ≈ 502.5 KB for W = 335). Sequence bits for W = 64: $2^{k-1} \geq 64$ → **7 bits**.

- **Choice:** Selective Repeat with W = 335 (10-bit numbers) for the WAN – it is the only option that fills the link, and 0.5 MB of buffer is cheap; Go-Back-N is acceptable on the LAN where a $\approx 0.08$.

## TASK 10 – Find Where the Delay Comes From

### 1. Reading the traceroute (median of three probes)

| Hop | Address | Median RTT | Increase | Comment |
|---|---|---|---|---|
| 1 | 10.10.1.1 | 0.6 ms | +0.6 | Plant gateway (private address) |
| 2 | 100.64.0.1 | 3.1 ms | +2.5 | Carrier edge (100.64/10 shared address space) |
| 3 | 172.31.4.9 | 4.2 ms | +1.1 | One probe 38.2 ms – outlier |
| 4 | 103.21.8.17 | 11.8 ms | **+7.6** | **Largest jump – the long-haul WAN segment** |
| 5 | * * * | – | – | No reply to probes |
| 6 | 10.200.0.10 | 12.6 ms | +0.8 | Head-office server |

- **Propagation:** 400 km ÷ $2\times10^8$ m/s = **2 ms one way, 4 ms round trip** = 4 ÷ 12.6 = **31.7 %** of the measured 12.6 ms. The other ~8.6 ms is equipment processing, queuing, serialisation and the real cable route being longer than 400 km.
- **Hop 3 (38.2 ms once):** a single delayed probe – the router answers ICMP on its slow control plane (rate-limited, low priority), or hit a transient queue; hops 4 and 6 are not slower, so forwarding is fine. **Hop 5 (stars):** the router is configured/firewalled not to answer TTL-expired ICMP, yet it forwards traffic (hop 6 replied). **Neither is a fault.**
- **Most delay:** hop 4 – the 3 → 4 link adds ≈ 7.6 ms (≈ 60 % of the total), consistent with the 400 km carrier link.

### 2. My own measurement (to attach – run on a real connection)

Windows: tracert -d <server>  ping -n 20 <server>
Linux : traceroute -n <server>  ping -c 20 <server>
Wireshark: capture filter "icmp or tcp port 80" while running the commands; save as Task10.pcapng

| Layer / header | Fields to identify in my capture (fill in from my screenshot) |
|---|---|
| Ethernet II (L2) | Destination MAC: _____ Source MAC: _____ Type: 0x0800 |
| IPv4 (L3) | Source IP: _____ Destination IP: _____ TTL: ____ Protocol: ____ |
| ICMP / TCP (L4) | ICMP type/code: ____ or TCP ports ____ → ____, flags: ____ |
| Application (L7) | Protocol / payload: _____ |
| Ping result | min/avg/max = ____ / ____ / ____ ms, loss ____ % |
| Screenshots | [Insert tracert screenshot] [Insert ping screenshot] [Insert Wireshark screenshot with headers marked] |

### Alternative Concept Analysis – SNMP/NetFlow monitoring vs occasional ping/traceroute

| Problem in the traceroute | Occasional ping / traceroute | SNMP counters + NetFlow |
|---|---|---|
| 38.2 ms spike (hop 3) | Caught only if someone runs it at that second; a continuous ping/IP-SLA catches it within seconds | Polling averages (1–5 min) hide a single spike; shows queue drops/utilisation if a pattern develops; NetFlow shows which flows were present |
| Stars at hop 5 | Shows immediately but cannot tell harmless from broken | Interface and flow counters show traffic passing normally → proves no fault, no false alarm |
| Hop 4 high delay | Locates the hop on demand, in seconds | Utilisation/drops on the WAN link seen in 1–5 min; NetFlow names the heavy talker (e.g. CCTV vs robot traffic) |

Best practice: continuous SNMP + NetFlow for trends and fast alarms, with ping/traceroute for on-demand fault localisation.

## TASK 11 – Defend the Network Design

### 1. Recommended design, with numbers from Tasks 1–10

| Area | Medium / wireless | Ethernet settings | Error + flow control, with justification |
|---|---|---|---|
| Robot cell (85 m) | Cat6A S/FTP; no Wi-Fi | 1 Gbps full duplex, auto-neg ON, own VLAN, strict-priority QoS | CRC-32 FCS + TCP on a LAN with a = 0.08 (85.7 % even stop-and-wait, Task 9). Shannon 1,257 Mbps at 15 dB ≫ 41.3 Mbps of commands (Tasks 3–4). |
| Office (140 m) | Multimode fibre + Wi-Fi 6 (160 MHz) | 1 Gbps full duplex, office VLAN | CRC + TCP. Wi-Fi 6 limit 1,329.5 Mbps vs 30 Mbps/line need (Task 4). |
| Shop floor → data centre (550 m) | Single-mode fibre, 10 Gbps | 10 Gbps; jumbo (9,000 B) only in CCTV/storage VLAN | Plant load 1,265.92 Mbps = 126.6 % of 1 Gbps → 12.7 % of 10 Gbps (Task 1); jumbo gives 6.1× fewer frames (Task 6). |
| AGVs / mobile | Private 5G URLLC | – | FEC (LDPC/polar) + HARQ: ARQ waits 40 ms > 10 ms deadline (Task 8). 3 hand-overs = 60 ms vs 5.85 s on Wi-Fi (Task 5). |
| Sensors | Zigbee/LoRa slow sensors; vibration on mMTC/wired | – | Hamming FEC: 5 check bits (31.25 %), mean delay 0.008 ms vs 0.65 ms ARQ (Task 8). 240 kbps = 48× a LoRa gateway (Task 5). |
| Head-office link (400 km) | Leased line 10 Gbps (2 carriers) | PPP/HDLC framing | Selective Repeat, W = 335, 10-bit sequence numbers; stop-and-wait would give only 2.99 Mbps (Task 9). a = 166.67, propagation 2 ms (Task 10). |

### 2. The two biggest remaining risks

- **Real-time wireless reliability:** even 5G hand-over takes 20 ms (> 10 ms deadline) – 6 missed commands per 600 s trip – and welding/metal can degrade coverage. Mitigate with overlapping cells, dual connectivity and AGV on-board safe-stop.
- **Single points of failure and capacity:** one WAN line and star uplinks – a cut isolates the plant (Task 6); 1 Gbps already saturates at 126.6 %. Mitigate with a second carrier, ring (MRP) in the plant and 10 Gbps links.

### 3. Does it work for five plants?

- **Yes, with scaling.** 5 × 1,265.92 = 6,330 Mbps ≈ 6.3 Gbps in total, but only production data (not CCTV) crosses to the head office; 10 Gbps leased lines (W ≈ 335 frames) carry this. Use hierarchical addressing (a /21 per plant, /24 VLANs of ≤ 250 hosts for the 1,500 devices), the same standard-based equipment (Task 2) and local CCTV storage.

### Alternative Concept Analysis – whole plant on private 5G

| | Carry on 5G | Must stay on cable |
|---|---|---|
| Traffic (per plant, 8 lines) | Robots 8 × 32 = 256 Mbps (URLLC), sensors 8 × 0.24 = 1.9 Mbps (mMTC), AGVs, handhelds ≈ **258 Mbps** | CCTV 8 × 96 = 768 Mbps, office 8 × 30 = 240 Mbps ≈ **1,008 Mbps**, plus backbone, data-centre and WAN links |
| Why | Mobile, flexible, low-latency; hand-overs of 20 ms suit moving devices | Fixed, continuous, high-volume video would eat the cell capacity; Shannon/EMI margins and zero-hand-over make cable more deterministic |

258 + 1,008 = 1,266 Mbps (matches Task 1). Wired fall-back is kept for fixed robots where a loss of radio would stop production.

## TASK 12 – Re-evaluate and Adapt the Design

**Assumption for the AGV traffic:** each AGV receives a 200 B command every 10 ms (0.16 Mbps) and uploads one 4 Mbps camera/lidar stream → 4.16 Mbps per AGV.

### A. Impact analysis

| Area | Impact | Quantified |
|---|---|---|
| Traffic (Task 1) | 50 × 4.16 = 208 Mbps added | Plant 1,265.92 → **1,473.92 Mbps** = 147 % of a 1 Gbps uplink (+16.4 %) |
| Media | Uplink/backbone must be 10 Gbps single-mode; 5G radios/antennas added along the 1.2 km route | 1,473.92 Mbps = 14.7 % of 10 Gbps; 1 Gbps no longer viable |
| Wireless choice | Wi-Fi 6 cannot serve moving AGVs; 5G URLLC needed | Hand-overs 39 → 3 per trip (Task 5) |
| Error control | ARQ (40 ms retransmission) cannot meet 10 ms; radio needs FEC/HARQ | Hamming-type FEC 31 % overhead vs 40 ms spikes (Task 8) |

### B. Test the original design with 50 AGVs

| Measure (per 600 s trip) | Original Wi-Fi 6 | Private 5G |
|---|---|---|
| Hand-overs per AGV | 39 | 3 |
| Unreachable time per AGV | 39 × 150 ms = 5.85 s | 3 × 20 ms = 0.06 s |
| Missed 10 ms commands per AGV | 585 | 6 |
| For 50 AGVs per trip | 29,250 missed commands; 11.7 m blind travel each | 300 missed commands; 0.12 m blind travel each |
| System-wide hand-overs | 50 × 39 ÷ 600 = 3.25 per second | 0.25 per second |

**Result:** the original design **fails** – 99.0 % availability per AGV (5.85/600 = 0.975 % outage) cannot satisfy "never miss a command", and 802.11r (~30 ms) would still miss ≈ 117 commands per trip (Task 5). The 1 Gbps uplink also overloads (147 %).

### C. Adaptation: MODIFY (keep the wired core, change the wireless layer)

- **Retain:** Cat6A S/FTP in the robot cell, multimode/single-mode fibre, CRC-32 + TCP on wired links, full-duplex auto-negotiation, Selective Repeat W = 335 on the WAN, VLANs/QoS.
- **Modify:** 5G URLLC (5QI delay budget ≤ 10 ms) for AGVs (and optionally robots, with wired fall-back); Wi-Fi 6 for offices only; fibre backbone raised to 10 Gbps LAG; CCTV stays wired; FEC (LDPC/polar) + HARQ on 5G; 4–5 overlapping 300 m cells for conditional/dual-connectivity hand-over; on-board AGV safe-stop; ring (MRP) + second WAN carrier.

### D. Evidence-based defence

- **Numbers:** unreachable time falls from 5.85 s to 60 ms (**97.5× less**); missed commands per AGV from 585 to 6; blind travel from 11.7 m to 0.12 m; uplink load from 147 % of 1 Gbps to 14.7 % of 10 Gbps, leaving room for 5 plants (5 × 1,473.92 ≈ 7.4 Gbps in total).

- **Trade-offs:** 5G adds spectrum/core/radio cost and specialist skills, whereas Wi-Fi 6 is already installed and cheap; 5G still cannot guarantee zero misses (6 per trip), so redundancy (dual connectivity, safe-stop) is required; redesigning everything onto 5G (Task 11) is rejected because CCTV (768 Mbps) belongs on cable. **Decision: MODIFY – a full redesign is unnecessary.**
- This report with the evidence for Tasks 1–12: requirement tables, diagrams, calculations, Wireshark captures, measurements and evaluations.